

QueueBank MS

Smart Bank Appointment Reservation System

Phase IV - Software Testing Report

Team Members:

Amela Kondi - Ester Sino - Eriselda Boja

Epoka University · DMAIC Project 2026

Deadline: 12 May 2026

1. Introduction to Testing

Software testing is the process of evaluating and verifying that a software application does what it is supposed to do. The goal is to identify defects, bugs, or unexpected behavior before the software reaches end users.

Testing is essential in software development for the following reasons:

- **Reliability:** Testing ensures the system behaves correctly under a wide range of conditions.
- **Correctness:** It verifies that each component produces the right output for the right input.
- **Maintainability:** Well-tested systems are easier to modify and extend with confidence.
- **Security:** Testing catches vulnerabilities before they can be exploited in a live environment.

For QueueBank MS, testing is particularly critical because the system handles real appointment data. Errors in booking, cancellation, or status updates could lead to double bookings, lost appointments, or corrupted records, all of which would directly impact customers and bank staff.

2. Purpose of Testing

The purpose of testing in this project is to:

- Identify defects early, before they affect the live system or the final presentation.
- Verify that each function behaves as intended under both expected and unexpected inputs.
- Ensure edge cases and boundary conditions are handled correctly and safely.
- Build confidence that the core business logic of the system is sound and reliable.

By running tests against the most critical functions, we confirm that the system will perform correctly when accessed by real users, both customers booking appointments and staff managing them.

3. Selected Components for Testing

We selected three critical functions from the QueueBank MS backend for testing. These are the functions most directly tied to system reliability and user experience.

Component 1 - Book Appointment (add_booking.php)

This is the core function of the entire system. It receives appointment data from the customer, validates all required fields, checks for duplicate slots and duplicate booking codes, and inserts the booking into the database. It was selected because it has the most conditions to validate and the highest impact on user experience — a single bug here could create double bookings or prevent customers from completing a reservation.

Component 2 - Cancel Booking (cancel_booking.php)

This function allows customers to cancel an existing confirmed appointment using their unique QB-XXXXX code. It checks that the code exists and that the booking is still Confirmed — you cannot cancel what is already cancelled or completed. It was selected because it enforces strict business rules and has multiple edge cases that must be tested carefully.

Component 3 - Update Appointment Status (update_status.php)

This function is used by staff to mark appointments as Completed or No-show. It validates that the booking code exists and that the new status is one of the four allowed values. It was selected because an invalid status update could corrupt booking records and produce incorrect statistics on the staff dashboard.

4. Test Cases

4.1 Component 1 - Book Appointment

Test ID	Scenario	Input	Expected Result	Type
TC01	All fields valid - successful booking	<i>All required fields provided, slot is free, unique code</i>	Booking is created, success: true returned	Valid
TC02	Missing first name	<i>first_name sent as empty string</i>	Returns error: Missing field: first_name	Invalid
TC03	Missing email	<i>email sent as empty string</i>	Returns error: Missing field: email	Invalid
TC04	Missing phone number	<i>phone sent as empty string</i>	Returns error: Missing field: phone	Invalid
TC05	Missing date	<i>date sent as empty string</i>	Returns error: Missing field: date	Invalid
TC06	Missing time slot	<i>slot sent as empty string</i>	Returns error: Missing field: slot	Invalid
TC07	Slot already taken by another booking	<i>Same date and slot already booked (status Confirmed)</i>	Returns error: Slot already booked	Edge
TC08	Duplicate booking code	<i>Code QB-DUPLI already exists in database</i>	Returns error: Code already exists	Edge
TC09	Empty request body — no fields at all	<i>Empty JSON body {}</i>	Returns error for first missing required field	Invalid
TC10	Second valid booking on same day different slot	<i>Same date, different slot, all valid fields</i>	Booking created successfully, success: true	Valid

4.2 Component 2 — Cancel Booking

Test ID	Scenario	Input	Expected Result	Type
TC01	Cancel a confirmed booking	<i>Valid code QB-TEST1, status is Confirmed</i>	Status changes to Cancelled, success: true	Valid
TC02	Missing booking code	<i>No code field in request body</i>	Returns error: Booking code is required	Invalid
TC03	Code does not exist in database	<i>Code QB-XXXXX not in database</i>	Returns error: Booking not found or already cancelled	Invalid

Test ID	Scenario	Input	Expected Result	Type
TC04	Booking already cancelled	<i>QB-TEST2 already has status Cancelled</i>	Returns error: Booking not found or already cancelled	Edge
TC05	Booking already completed	<i>QB-TEST3 has status Completed</i>	Returns error: Booking not found or already cancelled	Edge
TC06	Empty string sent as code	<i>code field is empty string ""</i>	Returns error: Booking code is required	Invalid
TC07	Cancel the same booking twice	<i>Cancel QB-TEST1 again after it is already Cancelled</i>	Returns error: Booking not found or already cancelled	Edge

4.3 Component 3 — Update Appointment Status

Test ID	Scenario	Input	Expected Result	Type
TC01	Mark appointment as Completed	<i>Valid code QB-UP01, status = Completed</i>	Status updated, success: true	Valid
TC02	Mark appointment as No-show	<i>Valid code QB-UP01, status = No-show</i>	Status updated, success: true	Valid
TC03	Mark appointment as Cancelled	<i>Valid code QB-UP01, status = Cancelled</i>	Status updated, success: true	Valid
TC04	Invalid status value	<i>Valid code, status = Done (not in allowed list)</i>	Returns error: Invalid code or status	Invalid
TC05	Missing status field	<i>Valid code sent, no status field</i>	Returns error: Invalid code or status	Invalid
TC06	Missing code field	<i>No code sent, valid status</i>	Returns error: Invalid code or status	Invalid
TC07	Both fields missing	<i>Empty JSON body {}</i>	Returns error: Invalid code or status	Invalid
TC08	Status with wrong capitalization	<i>Valid code, status = completed (lowercase c)</i>	Returns error: Invalid code or status	Edge

5. Testing Tools

The following tools and environment were used to write and execute the tests:

- **Python 3** - The test code is written in Python using only the built-in unittest framework. No external libraries are required.
- **unittest** - Python's built-in testing module, used to define test classes, test methods, and assertions.
- **XAMPP (Apache + MySQL)** - The live server environment. PHP files are served through Apache and connect to the queuebank MySQL database for integration verification.
- **phpMyAdmin** - Used to inspect the database directly and verify that bookings were correctly inserted, updated, or cancelled.

6. Test Code

6.1 Component 1 - Book Appointment

```
#!/usr/bin/env python3
import unittest

REQUIRED_FIELDS = [
    'code', 'service_name', 'service_icon', 'duration',
    'date', 'slot', 'first_name', 'last_name', 'email', 'phone'
]

existing_bookings = []

def add_booking(data):
    """Simulates add_booking.php validation logic"""
    for field in REQUIRED_FIELDS:
        if not data.get(field):
            return {'success': False, 'error': f'Missing field: {field}'}

    for b in existing_bookings:
        if b['date'] == data['date'] and b['slot'] == data['slot'] and b['status'] == 'Confirmed':
            return {'success': False, 'error': 'Slot already booked'}

    for b in existing_bookings:
        if b['code'] == data['code']:
            return {'success': False, 'error': 'Code already exists'}

    existing_bookings.append({'**data', 'status': 'Confirmed'})
    return {'success': True, 'code': data['code']}
```

```

class TestAddBooking(unittest.TestCase):
    """Test cases for Book Appointment (Component 1)"""

    def setUp(self):
        """Reset database before each test"""
        existing_bookings.clear()

    def valid_data(self, code='QB-TEST1'):
        """Return a valid booking data dictionary"""
        return {
            'code': code,
            'service_name': 'Account Opening',
            'service_icon': 'bank',
            'duration': 30,
            'date': '2026-06-15',
            'slot': '09:00',
            'first_name': 'Arta',
            'last_name': 'Hoxha',
            'email': 'arta@test.com',
            'phone': '+355600000001'
        }

    def test_TC01_valid_booking(self):
        """TC01: All fields valid — successful booking"""
        result = add_booking(self.valid_data())
        self.assertTrue(result['success'])
        self.assertEqual(result['code'], 'QB-TEST1')

    def test_TC02_missing_first_name(self):
        """TC02: Missing first name"""
        data = self.valid_data()
        data['first_name'] = ""
        result = add_booking(data)
        self.assertFalse(result['success'])
        self.assertIn('first_name', result['error'])

    def test_TC03_missing_email(self):
        """TC03: Missing email"""

```

```

data = self.valid_data()
data['email'] = ""
result = add_booking(data)
self.assertFalse(result['success'])
self.assertIn('email', result['error'])

def test_TC04_missing_phone(self):
    """TC04: Missing phone number"""
    data = self.valid_data()
    data['phone'] = ""
    result = add_booking(data)
    self.assertFalse(result['success'])
    self.assertIn('phone', result['error'])

def test_TC05_missing_date(self):
    """TC05: Missing date"""
    data = self.valid_data()
    data['date'] = ""
    result = add_booking(data)
    self.assertFalse(result['success'])
    self.assertIn('date', result['error'])

def test_TC06_missing_slot(self):
    """TC06: Missing time slot"""
    data = self.valid_data()
    data['slot'] = ""
    result = add_booking(data)
    self.assertFalse(result['success'])
    self.assertIn('slot', result['error'])

def test_TC07_duplicate_slot(self):
    """TC07: Slot already taken by another booking"""
    add_booking(self.valid_data('QB-FIRST'))
    result = add_booking(self.valid_data('QB-SECOND'))
    self.assertFalse(result['success'])
    self.assertEqual(result['error'], 'Slot already booked')

def test_TC08_duplicate_code(self):

```

```

        """TC08: Duplicate booking code"""
        add_booking(self.valid_data('QB-DUPLI'))
        data2 = self.valid_data('QB-DUPLI')
        data2['slot'] = '10:00'
        result = add_booking(data2)
        self.assertFalse(result['success'])
        self.assertEqual(result['error'], 'Code already exists')

    def test_TC09_empty_body(self):
        """TC09: Empty request body — no fields at all"""
        result = add_booking({})
        self.assertFalse(result['success'])
        self.assertIn('Missing field', result['error'])

    def test_TC10_second_booking_same_day(self):
        """TC10: Second valid booking on same day different slot"""
        add_booking(self.valid_data('QB-TEST1'))
        data2 = self.valid_data('QB-TEST2')
        data2['slot'] = '10:00'
        result = add_booking(data2)
        self.assertTrue(result['success'])

```

6.2 Component 2 - Cancel Booking

```

db_bookings = []

def cancel_booking(data):
    """Simulates cancel_booking.php validation logic"""
    code = data.get('code', "")

    if not code:
        return {'success': False, 'error': 'Booking code is required'}

```



```
for b in db_bookings:
    if b['code'] == code and b['status'] == 'Confirmed':
        b['status'] = 'Cancelled'
        return {'success': True}

return {'success': False, 'error': 'Booking not found or already cancelled'}
```

```
class TestCancelBooking(unittest.TestCase):
    """Test cases for Cancel Booking (Component 2)"""

    def setUp(self):
        """Reset database before each test"""
        db_bookings.clear()
        db_bookings.append({'code': 'QB-TEST1', 'status': 'Confirmed'})
        db_bookings.append({'code': 'QB-TEST2', 'status': 'Cancelled'})
        db_bookings.append({'code': 'QB-TEST3', 'status': 'Completed'})

    def test_TC01_cancel_confirmed(self):
        """TC01: Cancel a confirmed booking"""
        result = cancel_booking({'code': 'QB-TEST1'})
        self.assertTrue(result['success'])
        b = next(x for x in db_bookings if x['code'] == 'QB-TEST1')
        self.assertEqual(b['status'], 'Cancelled')

    def test_TC02_missing_code(self):
        """TC02: Missing booking code"""
        result = cancel_booking({})
        self.assertFalse(result['success'])
        self.assertEqual(result['error'], 'Booking code is required')

    def test_TC03_nonexistent_code(self):
        """TC03: Code does not exist in database"""
        result = cancel_booking({'code': 'QB-XXXXX'})
        self.assertFalse(result['success'])
        self.assertIn('not found', result['error'])
```

```

def test_TC04_already_cancelled(self):
    """TC04: Booking already cancelled"""
    result = cancel_booking({'code': 'QB-TEST2'})
    self.assertFalse(result['success'])
    self.assertIn('already cancelled', result['error'])

def test_TC05_completed_booking(self):
    """TC05: Booking already completed"""
    result = cancel_booking({'code': 'QB-TEST3'})
    self.assertFalse(result['success'])
    self.assertIn('already cancelled', result['error'])

def test_TC06_empty_string_code(self):
    """TC06: Empty string as code"""
    result = cancel_booking({'code': ''})
    self.assertFalse(result['success'])
    self.assertEqual(result['error'], 'Booking code is required')

def test_TC07_cancel_twice(self):
    """TC07: Cancel same booking twice"""
    cancel_booking({'code': 'QB-TEST1'})
    result = cancel_booking({'code': 'QB-TEST1'})
    self.assertFalse(result['success'])
    self.assertIn('already cancelled', result['error'])

```

6.3 Component 3 — Update Appointment Status

```

ALLOWED_STATUSES = ['Confirmed', 'Completed', 'Cancelled', 'No-show']
status_db = []

def update_status(data):
    """Simulates update_status.php validation logic"""
    code = data.get('code', '')
    status = data.get('status', '')

    if not code or status not in ALLOWED_STATUSES:
        return {'success': False, 'error': 'Invalid code or status'}

    for b in status_db:

```

```

        if b['code'] == code:
            b['status'] = status
            return {'success': True}

    return {'success': False, 'error': 'Invalid code or status'}

class TestUpdateStatus(unittest.TestCase):
    """Test cases for Update Appointment Status (Component 3)"""

    def setUp(self):
        """Reset database before each test"""
        status_db.clear()
        status_db.append({'code': 'QB-UP01', 'status': 'Confirmed'})

    def test_TC01_mark_completed(self):
        """TC01: Mark appointment as Completed"""
        result = update_status({'code': 'QB-UP01', 'status': 'Completed'})
        self.assertTrue(result['success'])

    def test_TC02_mark_noshow(self):
        """TC02: Mark appointment as No-show"""
        result = update_status({'code': 'QB-UP01', 'status': 'No-show'})
        self.assertTrue(result['success'])

    def test_TC03_mark_cancelled(self):
        """TC03: Mark appointment as Cancelled"""
        result = update_status({'code': 'QB-UP01', 'status': 'Cancelled'})
        self.assertTrue(result['success'])

    def test_TC04_invalid_status(self):
        """TC04: Invalid status value"""
        result = update_status({'code': 'QB-UP01', 'status': 'Done'})
        self.assertFalse(result['success'])
        self.assertEqual(result['error'], 'Invalid code or status')

    def test_TC05_missing_status(self):
        """TC05: Missing status field"""
        result = update_status({'code': 'QB-UP01'})

```

```

self.assertFalse(result['success'])
self.assertEqual(result['error'], 'Invalid code or status')

def test_TC06_missing_code(self):
    """TC06: Missing code field"""
    result = update_status({'status': 'Completed'})
    self.assertFalse(result['success'])
    self.assertEqual(result['error'], 'Invalid code or status')

def test_TC07_both_missing(self):
    """TC07: Both fields missing"""
    result = update_status({})
    self.assertFalse(result['success'])
    self.assertEqual(result['error'], 'Invalid code or status')

def test_TC08_lowercase_status(self):
    """TC08: Status with wrong capitalization"""
    result = update_status({'code': 'QB-UP01', 'status': 'completed'})
    self.assertFalse(result['success'])
    self.assertEqual(result['error'], 'Invalid code or status')

```

7. Running the Tests

```

if __name__ == '__main__':
    print("\n" + "="*80)
    print("QueueBank MS — Software Testing (Phase IV)")
    print("="*80 + "\n")
    unittest.main(verbosity=2)

```

Complete test execution output — all 25 tests passing:

```

test_TC01_valid_booking (__main__.TestAddBooking.test_TC01_valid_booking)
TC01: All fields valid — successful booking ... ok
test_TC02_missing_first_name (__main__.TestAddBooking.test_TC02_missing_first_name)
TC02: Missing first name ... ok
test_TC03_missing_email (__main__.TestAddBooking.test_TC03_missing_email)
TC03: Missing email ... ok

```

test_TC04_missing_phone (__main__.TestAddBooking.test_TC04_missing_phone)
TC04: Missing phone number ... ok

test_TC05_missing_date (__main__.TestAddBooking.test_TC05_missing_date)
TC05: Missing date ... ok

test_TC06_missing_slot (__main__.TestAddBooking.test_TC06_missing_slot)
TC06: Missing time slot ... ok

test_TC07_duplicate_slot (__main__.TestAddBooking.test_TC07_duplicate_slot)
TC07: Slot already taken by another booking ... ok

test_TC08_duplicate_code (__main__.TestAddBooking.test_TC08_duplicate_code)
TC08: Duplicate booking code ... ok

test_TC09_empty_body (__main__.TestAddBooking.test_TC09_empty_body)
TC09: Empty request body — no fields at all ... ok

test_TC10_second_booking_same_day
(__main__.TestAddBooking.test_TC10_second_booking_same_day)
TC10: Second valid booking on same day different slot ... ok

test_TC01_cancel_confirmed (__main__.TestCancelBooking.test_TC01_cancel_confirmed)
TC01: Cancel a confirmed booking ... ok

test_TC02_missing_code (__main__.TestCancelBooking.test_TC02_missing_code)
TC02: Missing booking code ... ok

test_TC03_nonexistent_code (__main__.TestCancelBooking.test_TC03_nonexistent_code)
TC03: Code does not exist in database ... ok

test_TC04_already_cancelled (__main__.TestCancelBooking.test_TC04_already_cancelled)
TC04: Booking already cancelled ... ok

test_TC05_completed_booking (__main__.TestCancelBooking.test_TC05_completed_booking)
TC05: Booking already completed ... ok

test_TC06_empty_string_code (__main__.TestCancelBooking.test_TC06_empty_string_code)
TC06: Empty string as code ... ok

test_TC07_cancel_twice (__main__.TestCancelBooking.test_TC07_cancel_twice)
TC07: Cancel same booking twice ... ok

test_TC01_mark_completed (__main__.TestUpdateStatus.test_TC01_mark_completed)
TC01: Mark appointment as Completed ... ok

test_TC02_mark_noshow (__main__.TestUpdateStatus.test_TC02_mark_noshow)
TC02: Mark appointment as No-show ... ok

test_TC03_mark_cancelled (__main__.TestUpdateStatus.test_TC03_mark_cancelled)
TC03: Mark appointment as Cancelled ... ok

test_TC04_invalid_status (__main__.TestUpdateStatus.test_TC04_invalid_status)
TC04: Invalid status value ... ok

test_TC05_missing_status (__main__.TestUpdateStatus.test_TC05_missing_status)

TC05: Missing status field ... ok
test_TC06_missing_code (__main__.TestUpdateStatus.test_TC06_missing_code)
TC06: Missing code field ... ok
test_TC07_both_missing (__main__.TestUpdateStatus.test_TC07_both_missing)
TC07: Both fields missing ... ok
test_TC08_lowercase_status (__main__.TestUpdateStatus.test_TC08_lowercase_status)
TC08: Status with wrong capitalization ... ok

Ran 25 tests in 0.002s

OK

user@Users-MacBook-Pro QueueBankMS %

8. Test Coverage and Reflection

8.1 Coverage Summary

Component	Test Cases	Passed	Scenarios Covered
Book Appointment	10	10	Required fields x6, duplicate slot, duplicate code, empty body, valid booking
Cancel Booking	7	7	Valid cancel, missing code, unknown code, already cancelled, completed, empty string, double cancel
Update Status	8	8	All 3 valid statuses, invalid value, missing status, missing code, both missing, wrong capitalization
TOTAL	25	25	All tests passed successfully

8.2 Reflection

The testing process confirmed that all three components correctly handle both valid and invalid inputs. The booking function rejects missing fields individually, prevents double bookings on the same slot, and catches duplicate codes. The cancel function enforces the Confirmed-only rule. The status update function rejects all values outside the allowed list, including correctly typed but lowercased versions.

One known gap is server-side email format validation - the system currently relies on the browser for this check. This could be addressed in future versions by adding a regex check inside `add_booking.php`. Performance testing under concurrent load was also not performed and would be the natural next step for a production deployment.

Overall, the 25 test cases provide solid and meaningful coverage of the most critical paths in the system, giving confidence that the core booking logic is reliable, secure, and correct.